Validating Completeness in Software Requirements Specifications using Large Language Models

Bachelor's Thesis of

Yavuz Karaca

At the KIT Department of Informatics
KASTEL – Institute of Information Security and Dependability

First examiner:	Prof. Dr. Ralf H. Reussner
Second examiner:	Prof. Dr.-Ing. Anne Koziolek
First advisor:	Lars König M.Sc.
Second advisor:	Prof. Dr. Ralf H. Reussner

24. June 2024 – 30. September 2024

Karlsruher Institut für Technologie
Fakultät für Informatik
Postfach 6980
76128 Karlsruhe

I utilized ChatGPT-4 strictly for language assistance, such as correcting spelling and grammar, and improving sentence structure without altering the original meaning. Its use was confined to linguistic adjustments, without any engagement in fact retrieval. I independently created all the text, including the factual content and critical analyses, before employing the AI assistant for these linguistic refinements.

Validating Completeness in Software Requirements Specifications using Large Language Models (Bachelor's Thesis)

I declare that I have developed and written the enclosed thesis completely by myself. I have not used any other than the aids that I have mentioned. I have marked all parts of the thesis that I have included from referenced literature, either in their original wording or paraphrasing their contents. I have followed the by-laws to implement scientific integrity at KIT.

Karlsruhe, 30. September 2024

. . . . . .

(Yavuz Karaca)

Abstract

This thesis explores the use of Large Language Models (LLMs) for validating the completeness of Software Requirements Specifications (SRS). Traditional SRS validation is a manual, time-consuming process that often results in missed requirements, potentially leading to project failures. This study leverages LLMs to automate the identification of incompleteness within SRS documents, focusing on completeness types defined by IEEE standards and Kuchta. The effectiveness of GPT-4 and LLaMA3 in identifying instances of incompleteness was evaluated through a series of experiments using different datasets and various prompting techniques. The F1 score was employed as the primary metric to assess performance.

Zusammenfassung

Diese Arbeit untersucht den Einsatz von Large Language Models (LLMs) zur Validierung der Vollständigkeit von Software-Anforderungsspezifikationen (SRS). Die herkömmliche Validierung von SRS ist ein manueller, zeitaufwändiger Prozess, der oft zu übersehenen Anforderungen führt, was potenziell zu Projektfehlern führen kann. In dieser Studie werden LLMs genutzt, um die Identifizierung von Unvollständigkeiten in SRS-Dokumenten zu automatisieren, wobei der Schwerpunkt auf den von den IEEE-Standards und Kuchta definierten Vollständigkeitstypen liegt. Die Wirksamkeit von GPT-4 und LLaMA3 bei der Erkennung von Unvollständigkeiten wurde durch eine Reihe von Experimenten mit verschiedenen Datensätzen und unterschiedlichen Prompting-Techniken bewertet. Der F1-Score wurde als primäre Metrik zur Leistungsbewertung verwendet.

Contents

Abstract **i**

Zusammenfassung **iii**

1. Introduction **1**

2. Foundations **3**
2.1. Large Language Models . . . 3
2.2. Prompt Engineering . . . 3
2.3. Requirement Engineering . . . 4
2.4. Completeness . . . 5

3. Related Work **7**

4. Completeness of Requirements Specifications **9**
4.1. Simple Completeness . . . 9
4.1.1. Formal Completeness . . . 10
4.1.2. Reference Completeness . . . 11
4.2. Explicit Semantic Completeness . . . 11
4.2.1. Functional Completeness . . . 11
4.2.2. Non-functional Completeness . . . 12
4.2.3. Constraint Completeness . . . 12
4.3. Completeness in the Presence of Implicit Domain Knowledge . . . 13

5. Evaluation Design **15**
5.1. Goal-Question-Metric Plan . . . 15
5.2. Datasets . . . 16
5.3. Gold Standard . . . 19
5.4. Prompts . . . 21
5.5. Prototype . . . 24

6. Evaluation Results **27**
6.1. Exploration . . . 27
6.2. Ollama . . . 28
6.3. Main Evaluation . . . 29
6.4. Delimiter Difference . . . 30
6.5. Observations . . . 31

7. Discussion **33**

8. Conclusion **35**

Bibliography **37**

A. Appendix **41**

List of Figures

3.1. Completeness metrics according to the Kuchta [20]. 8
3.2. Overall SRS evaluation in the study from Krishna [19]. 8

5.1. (a) File view of the dataset, PURE [11]; (b) A snippet from WARC Tools Phase III - 2009; (c) A snippet from The MultiMahjong Project - 1999. Both examples retrieved from the dataset. 17
5.2. The Process of Establishing Gold Standard 19
5.3. Example Gold Standard Table, entries sorted by level 20
5.4. Instantiating LLM with LangChain 25
5.5. Invoking LLM with a prompt 25

6.1. Exploration Results: F1 Scores 28
6.2. Final Results: Metrics by Level throughout the Dataset 29
6.3. Final Results: Metrics for each Document 30
6.4. Comparing Delimiters in Dataset 31

A.1. Coma Delimiter Level Scores 41
A.2. Coma Delimiter Total Scores 42
A.3. Comparing Delimiters in Dataset: 2005 - Nenios 42
A.4. Comparing Delimiters in Dataset: 2024 - Topographical Projection Simulation 43
A.5. Comparing Delimiters in Ollama: a) above is coma, b) below is semicolon . 43

List of Tables

5.1. High-Priority Documents from the PURE Dataset 18

A.1. Prioritization of Documents in PURE Dataset 44

1. Introduction

Large Language Models (LLMs) are becoming increasingly integral to our daily lives, workplaces, and scientific endeavors. Evaluating the capabilities of these models and exploring their potential applications is crucial. One relevant application is in requirements engineering [19], a fundamental software engineering process that involves identifying, analyzing, documenting, and validating the requirements for a system under development [35]. Traditionally, clear objectives are outlined in a Software Requirements Specification (SRS) document to ensure all stakeholders are aligned [17].

According to IEEE, a good SRS should exhibit eight characteristics: correctness, unambiguity, completeness, consistency, ranking for importance and/or stability, verifiability, modifiability, and traceability [16]. Validating an SRS document is typically manual and time-consuming, which motivates the adoption of agile methodologies that often forego the creation of such detailed documents [20]. A possible solution to this would be employing LLMs as an automated validation tool. This raises the question: if LLMs are provided with both the guidelines that an SRS should follow and the document itself, can they accurately validate it?

The research of Krishna et al. [19] assesses the capabilities of LLMs in drafting SRS documents, comparing them to benchmarks set by human-generated SRS, and focusing on all the characteristics specified by IEEE. Despite this advancement, there remains a significant gap in the literature regarding the use of LLMs not for drafting but for the automated validation of existing documents, akin to grammar checking tools.

The comprehensive evaluation of LLMs in identifying violations of IEEE guidelines would encompass all aspects outlined in the IEEE guidelines [16]. However, this work focuses specifically on Completeness, as its absence can compromise the foundational certainty of a project [20]. Research supports that "incomplete requirements are frequent, difficult to detect, and a significant source of implementation defects" [28]. Generally, an SRS document is considered complete if it encompasses all significant requirements and responses to all realizable inputs [16]. A precise definition of completeness and it's types is provided in Section 4.

The thesis evaluates the capacity of LLMs in identifiyng instances of incomepleteness in SRS documents by using documents modified from the PURE [11] dataset. For this Gold Standard was created, containing instances of incompleteness both identified by me in the original documents and artificially introduced. More details on how the experiment was conducted are found in Chapter 5. How different prompts and models performed on different SRS documents and different types of incompleteness are found in Chapter 6. Finally several limitations are discussed in Chapter 7.

2. Foundations

2.1. Large Language Models

Large language models (LLMs) are Artificial Intelligence (AI) systems trained on large datasets to process and generate natural language [15]. With their parameter sizes and learning capabilities [5], they have driven advancements in natural language processing (NLP) in recent years [18]. They can generate human-like text, answer questions, summarize text, "infer from context" and even assist in creative writing or code generation tasks [15]. Some of the notable examples include OpenAI's ChatGTP, Meta's Llama, and Google's BERT and PaLM [15].

They are utilized in many fields, from education and history to mathematics, medicine, and physics [27]. They offer opportunities to enhance learning and teaching experiences for individuals at all educational levels [18]. Evaluating LLMs is crucial to understand their strengths, weaknesses, and ensure their reliability, especially as they evolve and expand into critical sectors [5].

2.2. Prompt Engineering

Efforts in software engineering have utilized LLMs for tasks such as code generation and enhancement, with emerging research focusing on how best to prompt these models for optimal task performance [39]. "Prompt engineering is a relatively new discipline that refers to the practice of developing and optimizing prompts to effectively utilize large language models, particularly in natural language processing tasks" [12]. The process can be compared to asking a librarian, where you state your information needs to retrieve knowledge from an extensive collection [12].

The optimal prompting strategy for LLMs varies depending on resources, the specific model, the usage scenario, and variations across model versions [38]. One common strategy mentioned in the study is Chain-of-Thought Reasoning, where the model provides answers as intermediate steps before determining the final answer [38]. The same study also reports benefits gained from formatting the prompt; specifically, "enclosing each separate response in XML tags allowed us to easily parse its answer despite the added complexity."

OpenAI provides guidelines for prompt engineering [34]. According to these guidelines, key instructions such as "use expert-level writing" and "work out its own solution before rushing to a conclusion" enhance the likelihood of achieving better results. Similarly, they

recommend a second prompt that asks "if it missed anything on previous passes". To illustrate the value of deliberate processing, consider this: if asked to multiply 17 by 28, you might not know it instantly but can work it out over time. Similarly, allowing models more time to "think" leads to fewer errors and more accurate responses. They confirm that encouraging a "chain of thought" approach enables the model to navigate toward correct answers more reliably.

2.3. Requirement Engineering

Requirements engineering (RE) focuses on defining the goals for a planned system, translating these goals into specific services and constraints, and designating responsibilities for these requirements to entities like humans, devices, and software systems [21]. This process includes "domain analysis, elicitation, specification, assessment, negotiation, documentation, and evolution" [21].

Requirements are formally, explicitly, and precisely documented as Software Requirements Specification (SRS) documents, which can span hundreds of pages and include thousands of requirements [8]. These documents play a vital role in managing the success of the software project [9].

According to IEEE, a good SRS primarily offers these key benefits: It establishes a mutual understanding between customers and suppliers about the software's functionalities, which is crucial for agreeing on the product's requirements. It significantly reduces development effort by identifying requirement issues early, thus minimizing later redesign and retesting. Additionally, the SRS provides a realistic basis for estimating costs and schedules. Finally, it serves as a baseline for validation and verification, ensuring that the software meets all specified requirements [16].

According to IEEE, a good SRS should be [16]:

1. Correct: "every requirement stated therein is one that the software shall meet"
2. Unambiguous: "every requirement stated therein has only one interpretation."
3. Consistent: "no subset of individual requirements described in it conflict."
4. Ranked for importance and/or stability: "each requirement has an identifier to indicate either the importance or stability of that particular requirement."
5. Verifiable: "every requirement stated therein is verifiable", meaning "there exists some finite cost-effective process with which a person or machine can check that the software product meets the requirement"
6. Modifiable: "its structure and style are such that any changes to the requirements can be made easily, completely, and consistently while retaining the structure and style"
7. Traceable: "the origin of each of its requirements is clear and if it facilitates the referencing of each requirement in future development or enhancement documentation"

8. Complete: detailed in the next section.

2.4. Completeness

Incompleteness "causes uncertainty of the project foundations" [20], which forces engineers to make assumptions when dealing with incomplete requirements [1]. "Poor and incomplete SRS and inadequate requirements management are among the main reasons for project failure" [1], therefore validating SRS completeness is vital.

According to IEEE, an SRS document is complete if it [16]:

1. Includes all significant requirements, such as functionality, performance, design constraints, attributes, or external interfaces.
2. Defines responses to all realizable input data in any conceivable situation.
3. Provides complete labels and references for all figures, tables, and diagrams, and defines all terms and units of measure.

3. Related Work

Bankauskaite et al. [3] use Model-Based Systems Engineering (MBSE) and Systems Modeling Language (SysML) to automate the completeness evaluation of SRS. Similar to our work, their research is concerned with the aspect of completeness in requirements engineering. They utilize SysML, a modeling language designed to create cohesive and consistent models, and the MBSE Grid to evaluate and improve the completeness of SRS, emphasizing early detection and correction of incompleteness to enhance overall project outcomes. The MBSE Grid is a matrix that organizes systems engineering into two main viewpoints across rows—defining the problem and proposing solutions—and four main aspects across columns: requirements, system structure, system behavior, and parameters. However, their approach diverges significantly from ours in methodology and tools as they do not utilize LLMs.

Kuchta [20] introduces three key metrics to evaluate SRS completeness as shown in Figure 3.1): Formal Completeness (FC) focuses on the extent to which the document template is filled (Template Completeness Factor) and the adequacy of definitions within the SRS (Definition Completeness Factor). Semantic Completeness (SC) is measured by the Missing Semantic Element Count, requiring expert assessment to identify and scale semantic gaps relative to the total semantic content. Reference Completeness (RC) evaluates the adequacy of both solution-oriented references and internal references required by the document's framework. These metrics offer a structured approach to assess the completeness.

His study, however, focuses on the broader impacts of quality requirements analysis and how meticulous attention to SRS consistency directly influences completeness and thereby the discovery of functional requirements [20]. In contrast to our work, the analysis is conducted manually without the use of LLMs.

Krishna et al. [19] demonstrate the usage of LLMs in drafting SRS for software development, exploring their ability to produce structured, accurate, and coherent documents. The study assesses GPT-4 and CodeLlama in drafting an SRS for a university club management system, comparing their outputs to human benchmarks. Results (see Figure 3.2) show GPT-4 can produce drafts comparable to entry-level engineers and provide constructive feedback, while CodeLlama was less effective. The research highlights LLMs' potential to reduce development time and boost productivity in software engineering [19]. While Krishna's research highlights how LLMs generate SRS documents, this thesis focuses on using LLMs to identify incompleteness in SRS documents.

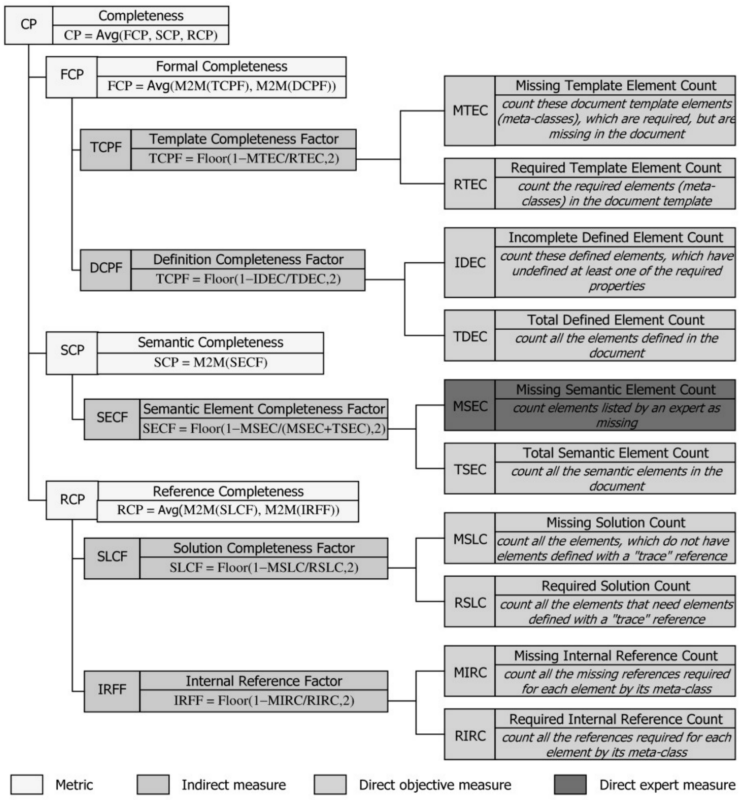

Figure 3.1.: Completeness metrics according to the Kuchta [20].

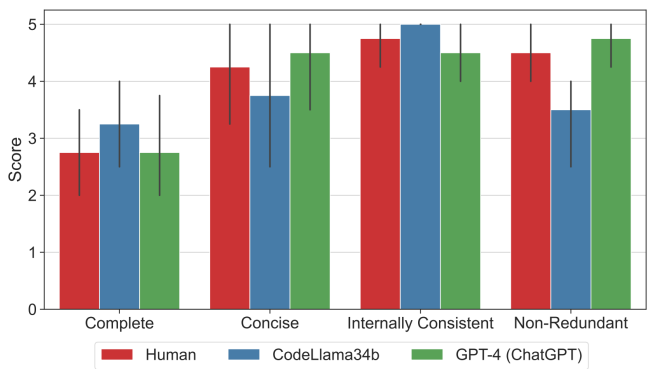

Figure 3.2.: Overall SRS evaluation in the study from Krishna [19].

4. Completeness of Requirements Specifications

In this thesis I define three levels of completeness to assess the capabilities of LLMs in identifying different types of completeness: Simple Completeness, presented in Section 4.1, Explicit Semantic Completeness, in Section 4.2 and Implicit Domain Knowledge Requiring Completeness in Section 4.3.

Simple Completeness involves identifying easily detectable gaps such as missing elements in lists, labels, and references without needing contextual understanding. Explicit Semantic Completeness focuses on identifying gaps in essential requirements like functionality, performance, and design constraints by evaluating the provided information in the document. Implicit Domain Knowledge Requiring Completeness involves identifying assumed but undocumented domain-specific knowledge that requires external understanding beyond what is explicitly stated in the document. The categorization is informed by foundational research in text comprehension, which emphasizes coherence and the cognitive processes involved in understanding text [37]. Just as these coherence types help readers understand and connect textual elements, the levels of completeness facilitate a structured approach to identifying gaps, ranging from simple to those requiring domain knowledge.

4.1. Simple Completeness

Simple Completeness comprises two key aspects: Formal Completeness 4.1.1 and Reference Completeness 4.1.2. Incompleteness at this level includes gaps, such as missing sections within document, elements within lists and glossaries, or the absence of references for related requirements or diagrams [20]. This corresponds to the violation of the third item in the IEEE guidelines defined in previous sections as "complete labels and references for all figures, tables, and diagrams, and defines all terms and units of measure" [16].

The key aspect of validating simple completeness is that it does not necessitate an understanding of the project or its context, thereby eliminating the need for deeper comprehension or interpretation [37]. Essentially, verifying simple completeness is comparable to conducting a macro level syntax analysis.

4.1.1. Formal Completeness

Formal Completeness, as defined by Kuchta [20], can be further categorized into two distinct types:

1. **Template Completeness:** This component refers to the extent to which all required sections and elements within a document template are present and accounted for. To validate template completeness, any missing elements required by the document template must be identified [20]. If a required element is skipped, it can be detected through enumeration, as the missing element creates a gap in the sequence unless it is the last element. Additionally, tables may be present but contain unfilled cells, which also indicates incompleteness. This applies to chapters as well. For instance, if a document lacks an entire chapter, such as an introduction or constraints this results in template incompleteness. Various templates exist for SRS documents, including those recommended by IEEE 830–1998 for Object-Oriented development, as well as V-Modell and Agile Development frameworks [2]. Despite differences, these templates share many common elements. Thus, a foundational template that encompasses all **essential** components should include the following items:

   - Introduction
   - Purpose (or Goals and Objectives)
   - Scope
   - Overview
   - Functional Requirements
   - Non-functional Requirements
   - Constraints
   - Glossary (or Definitions/Acronyms)

   Building on Kuchta's definition [20], this thesis extends the notion of template completeness beyond document-wide templates to include specific requirement-driven templates. If a specific requirement establishes a template, for example by stating "the program does the following three modes," then the items of this list can be validated the same way, and a missing item would result in a failure to fulfill the template. Thus, the concept of template completeness solely focuses on identifying gaps by checking if all expected items are present and correctly matched, without any additional analysis.

2. **Definition Completeness:**

   The SRS may contain project-related terms, abbreviations, and units of measure. All of these should be briefly described and included in a separate chapter, such as a glossary. Definition completeness examines whether all required properties of elements are fully defined, identifying any elements with incomplete definitions. However, if the definitions are incorrect but present, this is no longer considered a violation of

definition completeness, as correction requires domain knowledge, which is discussed in subsection 4.3.

4.1.2. Reference Completeness

Kuchta [20] also defines Reference Completeness, which includes:

1. **Solution Completeness:** This component evaluates the completeness of the "trace" references that lead to solutions within the SRS. It includes specific parts of the SRS that explain the design, architecture, or implementation details necessary to achieve the requirements. These descriptions can cover algorithms, modules, subsystems, interfaces, data structures, and any other technical details that outline how the system will function and meet its specified requirements.
2. **Internal Reference:** This component evaluates whether all required internal references (e.g., cross-references within the document) are present and correctly linked. This ensures that the document is self-contained and that references within it lead to the correct sections or elements, facilitating ease of navigation and understanding.

4.2. Explicit Semantic Completeness

Explicit Semantic Completeness is defined as the extent to which a document thoroughly addresses and specifies all significant requirements. Incompleteness at this level manifests as gaps in essential requirements, such as functionality, performance, and design constraints, violating the IEEE guideline that a document should contain "all significant requirements, whether relating to functionality, performance, design constraints, attributes, or external interfaces" [16]. Additionally, this level involves interactions or dependencies between requirements that are not clearly defined, or where the behavior is only partially specified or completely unspecified. These gaps are identifiable solely based on the information provided in the document.

4.2.1. Functional Completeness

Functional Completeness evaluates whether all system functionalities and behaviors are thoroughly and clearly defined, adhering to the IEEE guideline that a document should include "definition of the responses of the software to all realizable classes of input data in all realizable classes of situations" [16].

By examining the functional requirements in the dataset, common examples of incompleteness were identified, including but not limited to:

1. **Under-specified Requirements:** Some requirements may be described only at a high level without sufficient detail. For example, "The system shall allow the user to upload files" may not specify file size limits, file types, or how to handle unsupported file types.
2. **Dependent Requirements:** Certain requirements may rely on other functionalities that are not specified, leading to incomplete implementations. For instance, "The system shall confirm successful file uploads" presumes an existing functionality that enables file uploads.
3. **Partial Behavior Specifications:** Incompleteness can occur when portions of a behavior are described, leaving out crucial states and actions of a system. For example, a "pause" function might be detailed without any requirements describing a "resume" function, or a multi-step approval process might lack guidelines on handling rejections and recovery procedures.
4. **Undefined Inputs:** The system might not specify behavior for all possible inputs, whether numerical, textual, or user interactions like clicks, especially under unusual conditions. For example, the requirement that "User should be able to set the speed of the simulation by entering a number" fails to define the behavior when zero or negative numbers are entered. Similarly, how the system responds to invalid inputs may not be specified. The IEEE guidelines on completeness emphasize this: "Note that it is important to specify the responses to both valid and invalid input values" [16].

4.2.2. Non-functional Completeness

Non-functional requirements are crucial for defining the quality and operational standards of a system, encompassing performance, usability, reliability, security, maintainability, and scalability [2]. Non-functional incompleteness arises when these critical aspects are not specified in detail. For example, a system might lack clear specifications on performance factors such as speed, response time, and throughput, or it may omit crucial details regarding user interface design and user experience enhancements. Similarly, specifications might not address reliability concerns like uptime and recovery procedures, or they may fail to outline necessary security measures including data protection, access control, and threat mitigation. Furthermore, the absence of guidelines on code maintainability, documentation, and how the system should scale with increased loads also contributes to non-functional incompleteness.

4.2.3. Constraint Completeness

Incompleteness in constraints could result in missing elements in critical areas:

1. **Interface Requirements:** This includes missing details on how different system modules will interact, insufficient specifications for interactions with external systems,

APIs, and third-party services, as well as gaps in user interface design, including incomplete information on screens, inputs, and outputs.

2. **Environmental Requirements:** There are gaps in specifications concerning the hardware required to run the software. Additionally, details on necessary operating systems, middleware, and other essential software dependencies might be lacking.
3. **Data Requirements:** There is an absence of necessary entities, attributes, or relationships, which are crucial for the system's data integrity and functionality.

4.3. Completeness in the Presence of Implicit Domain Knowledge

This level is related to the assumed knowledge about the application domain that may not be explicitly stated in the SRS document [13, 32]. Incompleteness at this level cannot be identified from the document alone, as it requires external knowledge about the application domain. Consider a wrong definition of project-related terminology. Identifying and correcting it would require knowledge of the project and its domain. This type of incompleteness arises also from overlooked or implicitly assumed functionalities and qualities that are vital to the application but not formally documented.

For example, in the development of a football video game, developers might implicitly assume that the game should include mechanisms to resolve tie games according to football rules, such as extra time. If these gameplay elements are not explicitly specified in the SRS, the game may lack these features, and adherence to real-world football dynamics that players expect.

Similarly, a trading platform might operate under the implicit assumption that it will execute trades within milliseconds to capitalize on fast-moving markets. If the need for low latency is not explicitly captured in the requirements, the final product might fail to meet the critical operational standards of the trading sector. Such an oversight can impact the platform's competitiveness and ability to function effectively in a high-stakes environment where every millisecond counts.

5. Evaluation Design

The study employs the Goal-Question-Metric (GQM) Plan, a framework guiding evaluation by setting goals, forming questions, and using metrics to measure outcomes. To address the question of whether an LLM can accurately identify incompleteness in SRS documents, various documents were validated for completeness using LLMs. A gold standard was established by manually identifying incompleteness and with the help of GPT-4o, providing a baseline for comparison. Documents were collected from three datasets: a public requirements dataset and student projects from a practical course and a research project at KIT. Various prompt strategies, including zero-shot, few-shot, and chain of thought prompting, were used. To determine the best performing prompt, an initial exploration was conducted using a small dataset. Then the entire dataset was tested. The LLMs' results were compared to the gold standard, and their correctness was assessed using the F1 score.

5.1. Goal-Question-Metric Plan

To achieve a clearly structured approach, high traceability, and assessability of the work and results, the thesis followed the Goal-Question-Metric Plan (GQM Plan).

Goal: Validating completeness in SRS documents with LLMs.

Question: Given the guidelines that a SRS document should follow as well as a SRS document, can an LLM correctly identify all instances of incompleteness?

Metric: F1 score is the primary metric to assess the performance of LLMs in identifying incompleteness within SRS documents. The F1 score is defined as the harmonic mean of precision and recall [6]. Precision and recall are commonly used in information retrieval to evaluate the performance of classification systems and are calculated as follows [10]:

$$\text{Recall} = \frac{tp}{tp + fn}$$

$$\text{Precision} = \frac{tp}{tp + fp}$$

Here, tp, fp, and fn represent the number of true positives, false positives, and false negatives, respectively. In the context of validating SRS, a true positive occurs when an

instance of incompleteness is correctly identified. Conversely, a false positive happens when a non-existent violation is mistakenly flagged, and a false negative refers to a scenario where an actual violation goes undetected. To establish the ground truth, a Gold Standard was used, detailed in Section 5.3.

Thus, the F1 score is calculated as follows [25]:

$$F_1 = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} = \frac{2tp}{2tp + fp + fn}$$

This metric balances the trade-off between precision and recall, providing a comprehensive measure of the model's accuracy [14].

5.2. Datasets

Three different sources are used for the Dataset, which provide a wide range of different application domains, project sizes and types:

1. PUblic REquirements dataset (PURE) [11]
2. Projects from Students:
   a) Software Engineering in Practice (PSE)
   b) Research Assistance

The reason for including documents beyond PURE is to address concerns about whether LLMs have already been trained on this public data, as this could impact the performance of completeness validation. The documents from students are not publicly available or are from less popular sources of training data.

5.2.1. PUblic REquirements dataset (PURE)

PUblic REquirements dataset (PURE) comprising "79 publicly available requirements documents" with 34,268 sentences is used as the main data source [11]. This dataset is illustrated in Figure 5.1. Its variety and suitability for NLP tasks in requirements engineering [11], such as document structure assessment, make it ideal for evaluating the capacity of LLMs to identify and correct incompleteness in SRS documents.

The SRS documents from the PURE are selected based on the following criteria:

1. The document should have fewer than 50 pages to remain manageable.
2. All requirements should be described textually, without any information stated exclusively in the diagrams.

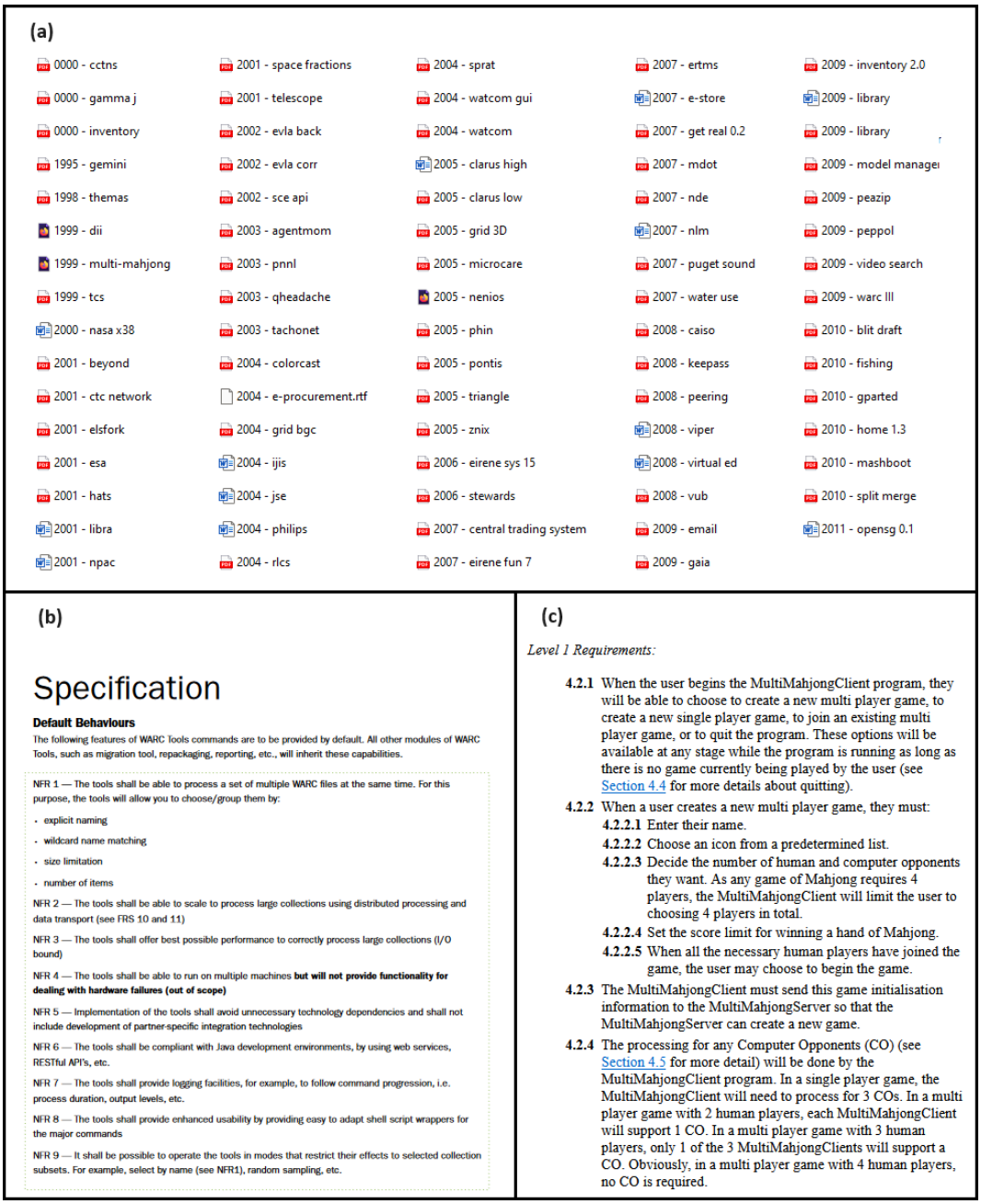

Figure 5.1.: (a) File view of the dataset, PURE [11]; (b) A snippet from WARC Tools Phase III - 2009; (c) A snippet from The MultiMahjong Project - 1999. Both examples retrieved from the dataset.

3. All functional and non-functional requirements must be listed separately and atomically, each labeled with a unique identifier.
4. The projects in the selected documents should span various application domains to ensure diversity.
5. Sections dedicated to functional and non-functional requirements should constitute at least one third of the document's length.

From this initial selection, documents are further sorted by readability and length to establish priority. Preference is given to documents with a clear format and shorter length. Lower priority documents are considered only if they contribute to diversity. Based on this, the following documents from the PURE dataset are given the highest priority:

Title	Used
1999 - multi-mahjong	No
2001 - esa	Yes
2005 - nenios	Yes
2005 - phin	No
2009 - email	No
2009 - library	No
2009 - video search	yes
2009 - warc III	No
2010 - blit draft	No
2010 - gparted	No
2010 - home 1.3	Yes
2010 - mashboot	No

Table 5.1.: High-Priority Documents from the PURE Dataset

The full categorization including lower priority documents can be found in Table A.1. Due to time constraints, only four of the documents could be manually inspected and thus used for the evaluation. This final selection was made randomly, as all documents were already deemed equally fitting.

5.2.2. Software Engineering (PSE)

"Software Engineering" (german Praxis der Software Entwicklung, shortly PSE) is a compulsory practical course at KIT where students work usually in teams of five to develop a software project over four to six months. They are mentored by doctoral researchers and are required to use the waterfall model. Therefore all projects produce a structured and detailed SRS document during the specification phase, making them fitting resources for evaluation because they are not publicly available.

To maintain consistency in language, only English documents were collected. However, since most students submit their work in German, this resulted in the collection of only three English documents.

5.2.3. Research Assistance

Research assistants play a role in supporting research and teaching activities at universities. These positions are typically filled by students who assist professors and researchers with

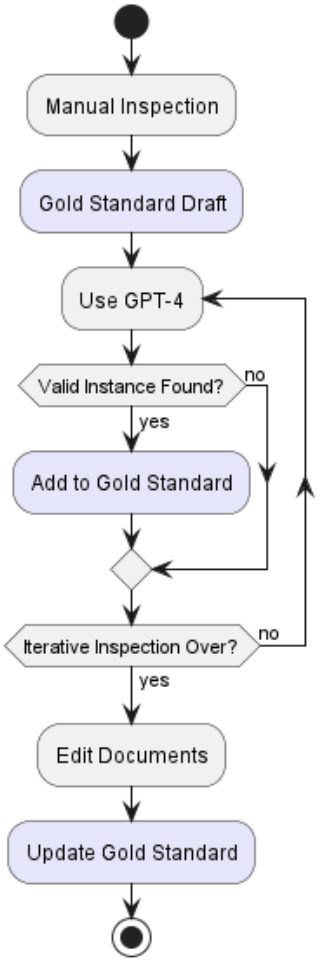

Figure 5.2.: The Process of Establishing Gold Standard

various tasks. These may include conducting experiments, managing data, assisting in the preparation of publications or developing software for research purposes.

In this context, a project done by me, which involves simulating a neurobiological model, was well-suited to serve as a data source due to its detailed SRS document.

5.3. Gold Standard

Instances of incompleteness in the selected documents from the dataset were recorded to establish a gold standard. Later, the documents were modified to introduce additional incompleteness, which was also included in the gold standard. The overview of this process is visualized in Figure 5.2.

I began with manually inspecting the documents to identify incompleteness. Each instance of incompleteness I could find was documented, including its type, underlying reasons,

	Label	Level	Issue	Correction	Native
1	FR-U-20	1	does not refer to behavior specifying requirement.	refer to FR-A-10	yes
2	FR-U-30	1	does not refer to behavior specifying requirement.	refer to PD20	yes
3	FR-U-50	1	does not refer to behavior specifying requirement.	refer to FR-A-10	yes
4	FR-U-60	1	does not refer to behavior specifying requirement.	refer to PD10	yes
5	FR-U-80	1	does not refer to behavior specifying requirement.	refer to FR-A-40	yes
6	FR-U-100	1	does not refer to behavior specifying requirement.	refer to PD30	yes
7	FR-A-50	1	requirement missing	...	no
8	PD30	1	does not fulfill the template of five elements	...	no
9	N/A	1	missing section on Non-functional requirements	...	no
10	Glossary	1	lacks definition of inference	Inference the process of using the knowledge of a trained n…	no
11	Section 2.2	2	does not specify the implied function of automatically scan…	System scans every 30 seconds to see if there is available …	yes
12	FR-A-30	2	does not specify recovery or program state when selection i…	... Improper Selection is undone and the previous state bef…	yes
13	FR-A-30	2	Does not specify the criteria for argument validity checks	The system shall check arguments for validity based on comp…	yes
14	FR-A-40	2	does not specify the behavior if user tries to start multip…	... or an inference is already started.	yes
15	FR-U-10	2	does not specify the default networks	LeNet and Alexnet are default and are always included in th…	yes
16	FR-U-40	2	does not specify if the listed hardware are all available a…	The system shall display an error message and prevent the s…	yes
17	FR-U-80	2	does not specify the behavior if the inference fails.	If inference fails the program returns to parameter selecti…	yes
18	FR-U-100	2	does not specify the behaviour if user tries to view result…	The system shall disable the 'View results' button until an…	yes
19	FR-U-110	2	does not refer to behavior specifying requirement.	refer to PD30	yes
20	FR-U-110	2	does not specify the behaviour if user tries to export resu…	The system shall disable the 'Export results' button until …	yes
21	FR-U-130	2	Does not specify the maximum number of images that can be s…	The system shall allow a maximum of 10 images to be selecte…	yes
22	PD20	2	does not specify the maximum number of neural networks that…	The system shall allow importing up to 10 neural networks s…	yes
23	PD40	2	does not specify the maximum size of the runtime log file.	The system shall limit the runtime log file size to 10MB an…	yes
24	Section 8	2	does not specify the performance requirements for the syste…	Inference should not take longer than 10 seconds for each i…	yes
25	FR-U-120	2	does not specify the behaviour if user tries abort while th…	The 'Abort' button will be only clickable when inference is…	no
26	PD10	2	does not specify the limit for images	...Max. 100 images per inference run	no
27	PD10	2	Does not specify the minimum height and width for input ima…	The height and width of input images should be at least 224…	yes
28	Section 5	2	does not specify the main implementation language	C++ as the main implementation language	no
29	PD20	3	does not specify what kind of topology is supported	The system shall allow importing only such topologies with …	yes
30	N/A	3	does not mention that neural networks should be trained for…	The imported neural network topologies should be specifical…	yes
31	Section 3.2	3	does not mention the loading animation while inference is r…	Loading animation will be shown after starting the inferenc…	no

Figure 5.3.: Example Gold Standard Table, entries sorted by level

and suggestions for improvement. After manual inspection, GPT-4 was used with various prompts, including those in Section 5.4. In an iterative process, potential issues were shared with the LLM, sometimes directing its focus to specific sections or even individual requirements. The LLMs' responses were carefully reviewed to identify any valid instances of incompleteness that had not been previously documented. These newly identified instances were subsequently included in the gold standard.

Then, the documents were manually transcribed into LaTeX format on Overleaf, excluding images but retaining tables, lists, and other textual representations. This approach allowed for the editing of documents originally in PDF or HTML format. Artificial instances of incompleteness were introduced into the document to enhance the diversity of incompleteness types and levels represented. These artificial instances were recorded alongside the native instances of incompleteness, with the "native" column marked as "no".

Additionally, blank pages between chapters, after the index, and before the glossary were removed to minimize the total page count, making the document easier for manage. To further reduce input size, test-related content were excluded from the documents in the PSE Dataset 5.2.2, as these requirements are not part of the foundational template outlined in subsection 4.1.1 and none of the other documents contained comparable sections.

The Gold Standard was then organized into a CSV file for manual comparison against LLM-generated results, as exemplified in Figure 5.3. This CSV file has the following columns:

- Label: The reference to the requirement or section where the incompleteness was found. For missing chapters "N/A" was used.

- Level: The level of the incompleteness, with "1" referring to Simple Incompleteness 4.1, "2" referring to Explicit Semantic Incompleteness 4.2 and "3" referring to Incompleteness due to Implicit Domain Knowledge 4.3.
- Issue: The cause of the incompleteness.
- Correction: An example of a requirement that would address the incompleteness.
- Native: Indicates if the incompleteness is from the original document ("Yes") or artificially introduced ("No").

5.4. Prompts

The modified documents were presented to the LLM with a variety of instructions, additional information, and guidelines, leading to the creation of multiple distinct prompts. As a foundational prompt for further techniques to build upon or modify, the following base prompt was employed:

Base Instruction

```
Identify all the instances of incompleteness in the following document by:
    1. Referring to the requirement by its unique identifier/label.
    2. Stating the issue.
    3. Providing an example of a requirement that addresses the
    identified incompleteness.
```

In LLMs, a "system message" sets the context and guides the model's behavior during a conversation, typically as the first prompt to instruct the LLM on its role and desired response format [24]. Figure 5.5 shows how system and user messages are used in the implementation. The following prompt was used as the system message, following Microsoft's prompt engineering guidelines [30]:

System Message: Default Role

```
You are an AI assistant that identifies incompleteness in Software
Requirement Specification documents.
```

Additionally, to enable easy comparison with the gold standard, another prompt was used as system message to structure the output as a CSV file [30]:

System Message: Output Format

```
Output Format:
Produce a CSV file with the following columns: "Label", "Issue", and
"Correction". Ensure that each cell value in the CSV file is
separated with a semicolon (;).
```

These configurations serve as the foundational setup. Additionally, various prompting techniques were employed, extending the base prompts:

- Zero-shot
- Few-shot
- Repeated Instructions
- Generated Knowledge
- Chain of Thought
- The Persona Pattern
- Providing the chapter on Completeness Types from this thesis

Each of these techniques is detailed in the following subsections, providing a comprehensive overview of their implementation.

5.4.1. Zero-Shot

Zero-Shot prompting asks the model to perform a task without providing any examples [42, 7]. This relies on the model's pre-trained knowledge. As such, the base configuration itself is an instance of Zero-Shot prompting.

5.4.2. Few-Shot

In few-shot prompting, a few examples of the desired input-output pairs are provided within the prompt to guide the model on how to respond [4]. "Few-shot prompting can be used as a technique to enable in-context learning where we provide demonstrations in the prompt to steer the model to better performance" [7].

The following excerpt from the prompt was used to provide examples for Few-Shot prompting:

Examples

```
Example Output:
Label;Issue;Correction
R10;Does not specify the format for international phone numbers;The system
shall accept phone numbers in the E.164 international format.
REQ-10 (2.1.4);Does not specify the maximum character length for user
comments;The system shall limit user comments to a maximum of 500
characters.
```

5.4.3. Repeated Instructions

Models may exhibit recency bias, where information presented at the end of a prompt disproportionately influences the output compared to information provided at the beginning [30]. To address this, repeating the instructions at the end of the prompt can influence the generated response. In the prototype, all prompts used to instruct the LLM are repeated after the document.

5.4.4. Generated Knowledge

Generated knowledge prompting involves first using a language model to generate relevant information and then incorporating this knowledge as additional context to enhance the model's response accuracy when answering a question [26, 7].

For this purpose, the prompt was iteratively refined until the output adhered precisely to the IEEE guidelines [16] as defined in Section 2.4. The final version of the prompt that achieved this is as follows:

Generate IEEE Guidelines

```
Provide the IEEE guidelines for Software Requirement Specification
documents about the Completeness aspect. According to IEEE, documents
should fulfill three things to be complete. What are these?
```

5.4.5. Chain of Thought

"A chain of thought is a series of intermediate natural language reasoning steps that lead to the final output" [41] and asking the LLM to tackle the task in this manner has been demonstrated to be effective [38].

The following prompt is used to replace the base instructions for Chain of Thought prompting:

Chain Of Thought

```
First, read the following document thoroughly and identify all the
instances of incompleteness.
Then, explain the reason of incompleteness for each instance referring to
their unique identifier.
After that, depending on the reason, provide an example of a requirement
that addresses the identified incompleteness.
Finally, take your time and think about these instances. Are they really
accurate and valid? Check if all the instances are included.
```

5.4.6. The Persona Pattern

"The Persona pattern gives the LLM a persona or role to play when generating output" [43]. For this purpose system level message was changed to:

System Message: Engineer Persona

```
You are an experienced Requirements Engineer who identifies incompleteness
in Software Requirement Specification documents.
```

5.4.7. Providing Completeness Types

Finally, the effect of providing the LLM with a comprehensive dictionary of Completeness types, as defined in Chapter 4 was examined. The prompt was created by simply copying the chapter from this thesis as plain text to be used alongside instructions.

5.5. Prototype

To assess the capabilities of LLMs', this study employed GPT-4, developed by OpenAI [33], and LLaMA3:8b, a model created by Meta AI with 8 billion parameters [29]. GPT-4 and LLaMA3 were selected for this study due to their capabilities [15, 38] and broad applicability in processing complex text data [40], making them ideal for the demands of validating SRS documents against IEEE standards.

The prototype was developed using `Python`, with the system built in `PyCharm`. The original dataset, modified documents, and gold standard tables were all included within the resources but only modified PDF documents were collected during inference. LangChain [22] was used to standardize interactions with both LLMs, ensuring consistent and efficient processing. For example, LLMs can be instantiated as illustrated in Figure 5.4 and then called uniformly, with specific implementation details abstracted away. To handle the input data, the `pypdfloader`

```
4 usages  Yavuz Karaca *
def create_llm_openai():
    """
    Creates an instance of the ChatOpenAI model with predefined settings for GPT-4o.
    """
    llm = ChatOpenAI(
        model="gpt-4o",
        temperature=0,
        max_tokens=None,
        timeout=None,
        max_retries=2,
        api_key=OPENAI_API_KEY
    )
    return llm
```

Figure 5.4.: Instantiating LLM with LangChain

```
Yavuz Karaca *
def engineer_persona(llm, doc):
    messages = [
        (
            "system", system_engineer_role() + output_format()
        ),
        ("user", instructions_base() + examples() + "\n".join([page.page_content for page in doc])),
    ]

    response = llm.invoke(messages)
    return response.content
```

Figure 5.5.: Invoking LLM with a prompt

[23] was used for splitting the documents into pages, similar to chunking, and loading, making the documents manageable for the LLMs. The best performing combination found during the exploration was used to invoke the LLMs along with the documents as input as shown in Figure 5.5. The results were then parsed and transformed into CSV files. The entire process was pipelined to ensure that both LLMs were invoked with all the desired combinations of prompts across the entire dataset.

The validation reports generated by the LLMs were manually compared with the gold standard, and the F1 score was calculated and saved as a CSV file. These scores were then visualized using `matplotlib`. Throughout development, `Git` was utilized for version control. The project's repository is available at GitLab[1], providing access to the full implementation details and artifacts.

[1]https://gitlab.kit.edu/kit/kastel/sdq/stud/abschlussarbeiten/masterarbeiten/yavuzkaraca

6. Evaluation Results

The evaluation was conducted in two stages. Initially, a small dataset (PSE) was tested to identify the best-performing prompt. Subsequently, the entire dataset was evaluated using the selected prompt, with attention to different levels of incompleteness instances.

The results from the LLM were compared with the gold standard, allowing each entry to be categorized as true positive (tp), false positive (fp), or false negative (fn). During the evaluation, if an incompleteness in the gold standard was recorded in a more general form than the instance identified by the LLM, all more detailed instances falling under this general category were counted individually, applying the same approach in cases of false positives. There were also instances of pseudo-incompleteness, where the identified issue represented a reasonable extension or an implementation decision. These were counted as false positives, with reasonable extensions classified as Level 3 false positives and implementation decisions as Level 2 false positives.

6.1. Exploration

An initial exploration using a small dataset (PSE) was conducted to identify the most effective prompt for evaluating incompleteness. Various prompts were tested to determine their ability to accurately identify instances of incompleteness within the dataset. Each prompt was assessed individually, using a few-shot learning approach when necessary, particularly for repeated instructions, generated knowledge, or completeness types. Performance was measured by calculating recall, precision, and F1 score.

As shown in Figure 6.1, the prompt combining completeness types and generated knowledge (GK + CT) consistently achieved the highest F1 scores across all three documents—2021 - init-v, 2022 - neutero, and 2023 - octo—demonstrating its effectiveness. In contrast, chain-of-thought (CoT) performed less effectively in both cases: with or without examples.

Interestingly, performance varied across documents even within the same prompt technique, indicating that document-specific factors influence validation effectiveness. The combined approach, which integrated all selected prompts, performed well but did not surpass the GK + CT combination.

Ultimately, the prompt that combined completeness types and generated knowledge was chosen as the default. This choice not only delivered the best performance but also aligned closely with the GQM question on completeness, which asks whether an LLM can correctly identify all instances of incompleteness in an SRS document, given the guidelines that

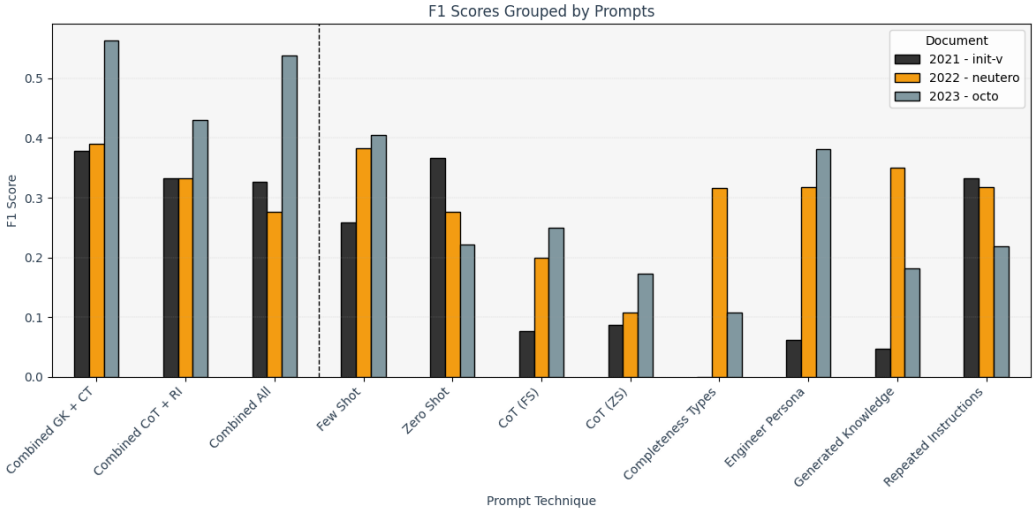

Figure 6.1.: Exploration Results: F1 Scores

such documents should follow. These guidelines are effectively conveyed through the completeness types and IEEE standards embedded within the generated knowledge, making this prompt particularly well-suited to the task.

6.2. Ollama

Meta's LLM, Llama3:8b, exhibited performance that was not sufficient for it to be evaluated in comparison with GPT-4o.

Figure (ref) visualizes the scores and outputs from a limited dataset.

The model consistently failed to produce the desired output format, leading to further tests to determine whether it could generate CSV files at all. Interestingly, it managed to create a CSV file for simple prompts like "Top ten places in Paris to visit." However, when tasked with summarizing an SRS document page by page, Llama3 repeatedly skipped certain pages. To investigate further, a specific prompt was used to summarize the consistently skipped sections, revealing that Llama3 struggled with loading the document properly, indicating technical difficulties rather than content comprehension issues.

- Provide data for skipping pages
- Provide few evaluations of the validation report from ollama to justify why we didnt continue with it

It's unclear why this phenomenon occurs, especially since no chunking was used. Chunking refers to dividing large text into smaller segments, and can be used to optimize the relevance of content retrieved from a vector database after embedding it using the LLM [36, 31]. In this case, plain text was extracted from the document and provided directly to the LLM. The

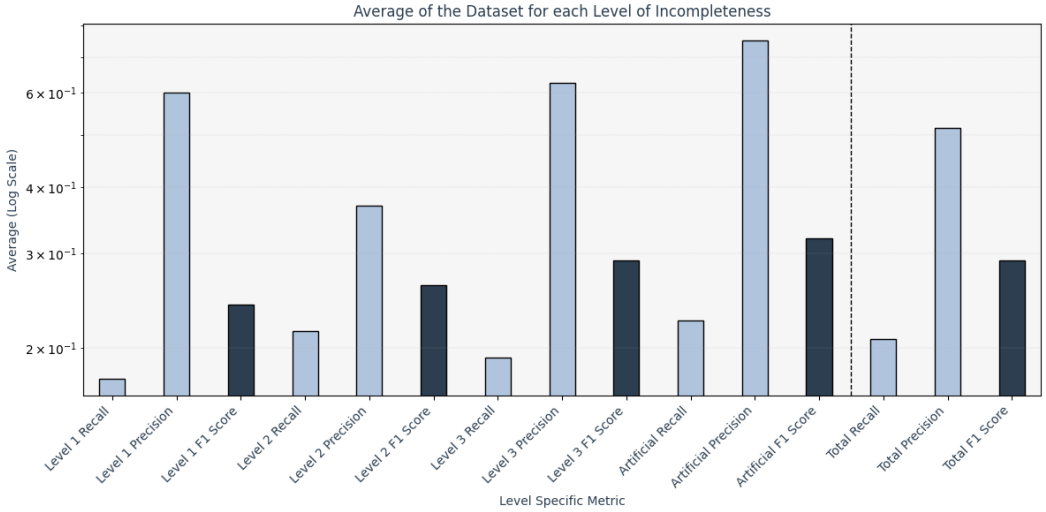

Figure 6.2.: Final Results: Metrics by Level throughout the Dataset

issue could be related to token size limitations or perhaps Llama3 requires embeddings and a vector database to effectively process such large texts.

- Maybe vector databese and embedding is necessary?

6.3. Main Evaluation

With the prompt combining completeness types and generated knowledge set as the default after the exploration phase, a comprehensive evaluation was conducted on the entire dataset to assess the model's effectiveness in validating completeness in SRS documents.

The results were grouped by Levels of Completeness. There are also two additional groups: "artificial," which shows performance only in the artificial instances of incompleteness, and "total," which includes all instances. It is important to note that artificial instances are included within the Levels, meaning they are not disjoint categories. For each category, recall, precision, and F1 score were calculated, with combined Levels visualized in Figure 6.2. Additionally, total recall, precision, and F1 score were plotted for each document, as shown in Figure 6.3.

The data in Figure 6.3 reveals that precision is consistently higher than recall across all documents, leading to the F1 scores also being higher than recall. Notably, documents such as 2001 - ESA, 2005 - Nenios, and 2024 - Topo Sim received the lowest validation scores. Interestingly, further analysis indicated that 2005 - Nenios and 2024 - Topo Sim could be validated more effectively with slight adjustments in the output format, as highlighted in the delimiter difference analysis, which discussed in next Section 6.4.

— This could be attributed to the inherent variability in the LLM's outputs, as each run may yield significantly different results.

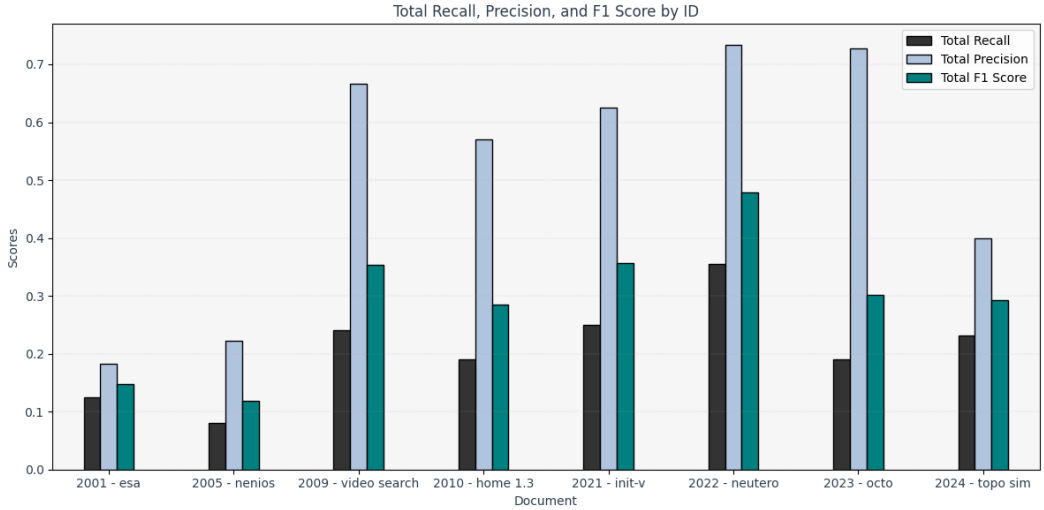

Figure 6.3.: Final Results: Metrics for each Document

This supports the idea that the poor performance observed may be due to the variability in prompt design and the nondeterministic nature of LLM outputs, rather than an inherent inability to validate these documents accurately. —

In the four documents that performed better—those other than 2001 - ESA, 2005 - Nenios, and 2024 - Topo Sim—precision values ranged between 0.5 and 0.7, indicating that the LLM correctly identified over half of the incompleteness instances. However, recall in these documents remained consistently below 0.4, showing that the LLM missed a significant portion of incompleteness instances. This underscores the necessity of human expertise to complement the LLM's capabilities in identifying instances that the model fails to capture.

The detailed breakdown in Figure 6.2 further supports these findings, showing that while the model is relatively effective at identifying certain levels of incompleteness, there are significant gaps, particularly in more complex or nuanced cases. This, and other observations are discussed in Section 6.5. The results reinforce the need for a hybrid approach, where LLMs are used in conjunction with human experts to ensure a comprehensive validation process.

6.4. Delimiter Difference

The variability in results is further influenced by the model's sensitivity to prompt design, as became evident when experimenting with different delimiters. Specifically, using a comma instead of a semicolon as a delimiter in CSV files led to substantial differences in the model's output, highlighting how even minor changes in prompt structure can impact the model's performance and reliability.

The following prompt was used instead of the default for output format prompt:

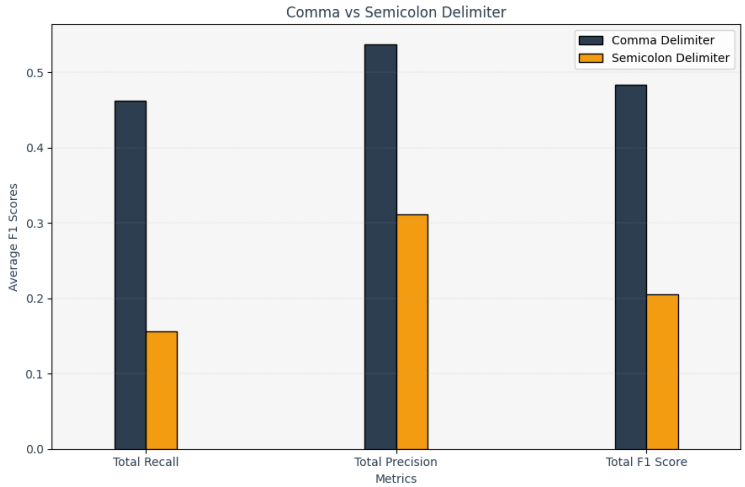

Figure 6.4.: Comparing Delimiters in Dataset

System Message: Comma Delimiter CSV Format

```
Output Format:
Produce a CSV file with the following columns: "Label", "Issue", and
"Correction".
```

Figure 6.4 illustrates the performance improvement in two specific documents that were previously among the worst validated: 2005 - esa and 2024 - topo sim. This enhancement was achieved simply by switching the delimiter. Additional figures providing further details on this comparison are included in the appendix.

The effects of changing delimiters were subsequently tested in Llama3, where similar results were observed, as shown in Figure A.5. It appears that LLMs generally perform better when using a comma as a delimiter, suggesting a consistent preference across different models. This could be due to the fact that most available training data predominantly used commas as delimiters, influencing the models' performance.

However GPT has a submodule for csv so the reason might be more complex:

- find source about subnet and add information about this to foundation

6.5. Observations

The evaluation of the LLM's performance revealed several limitations. These observations shed light on the model's challenges and areas where its effectiveness was notably constrained.

False Positives in Low-Incompleteness Documents

One notable pattern observed was that documents with little to no incompleteness tended to generate a higher number of false positives. This suggests that the model may struggle to distinguish between true incompleteness and well-documented cases where additional details are not strictly necessary. Instances of hallucination—where the model invents details that aren't present in the document—were also more prevalent in these scenarios.

Overly Strict Detection of Unlinked Details

Another observation was that when details were described elsewhere in the document but not explicitly linked, the model often flagged these as instances of incompleteness. While this is technically accurate when references are missing, it can sometimes be overly strict, especially when the connection between sections is intuitive. For example, the model flagged requirements as incomplete even when two related requirements were placed directly next to each other.

Sensitivity to Specific Incompleteness Types

Recurring patterns were observed in the types of entries identified as incomplete, particularly in three areas: formatting (such as input/output files or messages), missing sections (such as development or hardware environments), and error handling (such as behavior when invalid input is given or when something fails). These types of instances constituted the majority of what the LLM identified, often appearing in various variations of the same type across different sections of the document. This indicates that the model is particularly sensitive to these areas, likely influenced by the prompts used.

Challenges in Detecting Artificial Incompleteness

The evaluation revealed that the model had limited success in identifying artificial incompleteness, which was designed to test its ability to detect intentionally introduced errors. Specifically, artificially removing elements in tables, requirements randomly throughout the document, and items from lists were almost not identified at all, despite being straightforward cases. This suggests that the model struggles with basic, easily detectable issues, highlighting the need for further refinement in its training and prompt design.

Overfitting in Few-Shot Learning

Finally, it became evident that using too few examples in the few-shot learning approach led to overfitting. This was particularly apparent when only three examples were provided; nearly all validation reports incorrectly flagged "Glossary is missing," simply because it was included in the examples. This underscores the importance of a balanced and well-structured prompt to prevent the model from becoming overly specialized on a limited set of examples and misapplying learned patterns.

7. Discussion

The evaluation of LLMs for validating completeness in SRS documents revealed several important considerations and limitations. While the results demonstrated the potential of LLMs to assist in identifying incompleteness, there are key factors that must be discussed to fully understand the implications of this study.

Publicly Available Dataset and the Potential for LLMs to "Cheat"

One concern during the evaluation was that public requirements documents, which are openly accessible, might have been included in the training data of LLMs. This raises the possibility that the LLMs could "cheat" by recalling the original content of the documents when artificially introduced incompleteness was present. Essentially, the LLM might recognize what was missing simply because it had encountered the document before, rather than genuinely identifying incompleteness based on the prompt provided.

To mitigate this risk, student projects from the PSE dataset were also included in the evaluation. These projects were less likely to have been part of the LLMs' training data, thereby serving as a more reliable benchmark for assessing the models' true capability to detect incompleteness. The results in Figure 6.3 showed no indication that the LLMs were "cheating" on the public documents, as the models more effectively validated the student projects that were not publicly available. This suggests that the LLMs were genuinely analyzing the content rather than relying on memorized information.

Dataset Size and Range

It's important to note that student projects are not equivalent to the PURE dataset, which introduces a potential bias in the comparison. The challenge of finding enough student projects in English further limited the dataset size, making the comparison less comprehensive. Additionally, the number of documents in the dataset was constrained by the time-intensive nature of manual inspection. As a result, not all high-priority documents from the PURE dataset could be included in the gold standard. This limitation suggests that the evaluation may not fully reflect the models' performance across a broader and more diverse range of documents.

Limitations of Manual Inspection

Another important consideration is that I cannot guarantee that I identified all instances of incompleteness in the documents during my manual inspection. As a bachelor's student conducting this work independently, there is a possibility that some instances were overlooked. A review by multiple experts, who would manually inspect the documents to refine the gold standard, would likely uncover additional instances of incompleteness that I may have missed. Moreover, experts could better distinguish between true incompleteness and pseudo-incompleteness, particularly in those border cases. This more thorough inspection would likely result in different evaluations of the LLMs' effectiveness in detecting incompleteness.

Impact of GPT on Gold Standard Creation

Additionally, the process of using GPT during the creation of the gold standard introduces another layer of concern. Since GPT was involved in identifying certain types of incompleteness during the manual inspection, the gold standard may disproportionately reflect the kinds of incompleteness that GPT is naturally good at detecting. This overlap suggests that the model's perceived effectiveness might be inflated, as the evaluation may be biased toward the types of incompleteness that GPT was already predisposed to find.

However, despite GPT's role in helping the manual inspection, it surprisingly did not consistently reproduce many of these findings during the evaluation phase. This inconsistency is likely due to the fact that I employed a variety of prompts during the manual inspection, including those driven by personal suspicions, targeted focus on specific document sections, and follow-up questions. These conditions were not present during the evaluation, indicating that GPT's ability to identify incompleteness is highly dependent on the precise nature of the prompts used.

Role of Prompt Quality in Evaluation

This process highlights a critical factor: the quality and specificity of the prompts directly impact the effectiveness of the model's output. In other words, better prompts result in better detection of incompleteness. However, just as there is no guarantee that all instances of incompleteness were identified, there is also no guarantee that the optimal prompts were found and used. This uncertainty adds variability to the evaluation, as suboptimal prompts could result in a biased gold standard and an inaccurate assessment of the model's true potential.

8. Conclusion

This study aimed to explore the potential of using large language models (LLMs) to validate completeness in Software Requirements Specification (SRS) documents. The research involved evaluating various prompt designs on a dataset that included both public requirements and student projects. A gold standard was established through manual inspection, and the LLMs' outputs were compared against this standard using key metrics such as recall, precision, and F1 score.

The findings suggest that while LLMs show promise as a tool for supporting the validation process, it is still too early to consider them as a full replacement for manual validation. The models demonstrated some capability in identifying instances of incompleteness, particularly when guided by well-crafted prompts. However, they struggled with both complex and simple forms of incompleteness, especially when artificial or subtle errors were introduced.

LLMs can be particularly useful as a supporting tool in the validation process, helping to identify potential issues that might be overlooked in manual inspections. Interestingly, the study found that simple completeness, such as missing elements in tables or lists, might be more effectively validated using a syntax-based validation tool rather than relying solely on LLMs. This suggests that a hybrid approach, combining LLMs with more traditional validation tools, could enhance the overall validation process.

Future studies should expand on this research by using larger and more diverse datasets, incorporating a wider range of models, and thoroughly evaluating all aspects of the IEEE Guidelines. The prototype used in this work did not support the processing of images, such as sequence and activity diagrams, which are often critical components of SRS documents. Therefore, future studies should enable the model to receive these visual elements as well. Given the nondeterministic nature of LLMs, it will be important to generate results multiple times and consider the standard deviations in the evaluation. Additionally, involving experts in the manual inspection of the dataset will be crucial for creating the gold standard and ensuring a more accurate assessment of the models' performance.

In conclusion, while LLMs offer significant potential as a supporting tool in the validation of SRS documents, they are not yet ready to fully replace manual validation processes. To fully realize their potential, ongoing research is needed, including the integration of more sophisticated models and tools, especially those capable of addressing the full spectrum of completeness as defined by the IEEE guidelines. As LLMs continue to evolve and revolutionize various fields, their application in requirements engineering will likely become more refined, ultimately contributing to more efficient and reliable validation processes in the future.

Bibliography

[1] Özlem Albayrak, Hülya Kurtoglu, and Mert Biçakçi. "Incomplete Software Requirements and Assumptions Made by Software Engineers". In: *2009 16th Asia-Pacific Software Engineering Conference*. 2009, pp. 333–339. DOI: 10.1109/APSEC.2009.39.

[2] Helmut Balzert. *Lehrbuch der softwaretechnik: Basiskonzepte und requirements engineering*. Springer-Verlag, 2010.

[3] Jovita Bankauskaite et al. "An approach: Sysml-based automated completeness evaluation of the system requirements specification". In: *International Conference on Information Technologies (IVUS)*. 2018.

[4] Tom B. Brown et al. *Language Models are Few-Shot Learners*. 2020. arXiv: 2005.14165 [cs.CL]. URL: https://arxiv.org/abs/2005.14165.

[5] Yupeng Chang et al. "A Survey on Evaluation of Large Language Models". In: *ACM Trans. Intell. Syst. Technol.* 15.3 (Mar. 2024). ISSN: 2157-6904. DOI: 10.1145/3641289. URL: https://doi.org/10.1145/3641289.

[6] Nancy Chinchor. "MUC-4 evaluation metrics". In: MUC4 '92. McLean, Virginia: Association for Computational Linguistics, 1992, pp. 22–29. ISBN: 1558602739. DOI: 10.3115/1072064.1072067. URL: https://doi.org/10.3115/1072064.1072067.

[7] DAIR.AI. *Prompt Engineering Guide*. https://www.promptingguide.ai/. Accessed: 2024-08-21. 2024.

[8] Dev Dave et al. "Management of Implicit Requirements Data in Large SRS Documents: Taxonomy and Techniques". In: *SIGMOD Rec.* 51.2 (July 2022), pp. 18–29. ISSN: 0163-5808. DOI: 10.1145/3552490.3552494. URL: https://doi.org/10.1145/3552490.3552494.

[9] Vineet Dubey et al. "Making of Quality SRS for Requirement Engineering Process". In: ().

[10] Tom Fawcett. "An introduction to ROC analysis". In: *Pattern Recognition Letters* 27.8 (2006), pp. 861–874.

[11] Alessio Ferrari, Giorgio Oronzo Spagnolo, and Stefania Gnesi. "PURE: A Dataset of Public Requirements Documents". In: *2017 IEEE 25th International Requirements Engineering Conference (RE)*. 2017, pp. 502–505. DOI: 10.1109/RE.2017.29.

[12] L. Giray. "Prompt Engineering with ChatGPT: A Guide for Academic Writers". In: *Annals of Biomedical Engineering* 51 (2023), pp. 2629–2633. DOI: 10.1007/s10439-023-03272-4. URL: https://doi.org/10.1007/s10439-023-03272-4.

[13] Irit Hadar, Pnina Soffer, and Keren Kenzi. "The role of domain knowledge in requirements elicitation via interviews: an exploratory study". In: *Requirements Engineering* 19.2 (2014), pp. 143–159. ISSN: 1432-010X. DOI: 10.1007/s00766-012-0163-2. URL: https://doi.org/10.1007/s00766-012-0163-2.

[14] Purva Huilgol. *Precision and Recall in Machine Learning*. Accessed: 2024-06-16. 2020. URL: https://www.analyticsvidhya.com/blog/2020/09/precision-recall-machine-learning/.

[15] IBM. *Large Language Models*. Accessed: 2024-06-16. 2024. URL: https://www.ibm.com/topics/large-language-models.

[16] IEEE. *Recommended Practice for Software Requirements Specifications*. IEEE Std 830-1998. 1998.

[17] "IEEE Guide for Software Requirements Specifications". In: *IEEE Std 830-1984* (1984), pp. 1–26. DOI: 10.1109/IEEESTD.1984.119205.

[18] Enkelejda Kasneci. "ChatGPT for good? On opportunities and challenges of large language models for education". In: *Learning and Individual Differences* 103 (2023), p. 102274. ISSN: 1041-6080. DOI: https://doi.org/10.1016/j.lindif.2023.102274. URL: https://www.sciencedirect.com/science/article/pii/S1041608023000195.

[19] Madhava Krishna et al. *Using LLMs in Software Requirements Specifications: An Empirical Evaluation*. 2024. arXiv: 2404.17842 [cs.SE].

[20] Jarosław Kuchta. "Completeness and Consistency of the System Requirement Specification". In: *Position Papers of the 2016 Federated Conference on Computer Science and Information Systems*. 2016.

[21] Axel van Lamsweerde. "Requirements engineering in the year 00: a research perspective". In: *Proceedings of the 22nd International Conference on Software Engineering*. ICSE '00. Limerick, Ireland: Association for Computing Machinery, 2000, pp. 5–19. ISBN: 1581132069. DOI: 10.1145/337180.337184. URL: https://doi.org/10.1145/337180.337184.

[22] LangChain. *LangChain: Building Applications with LLMs*. Accessed: 2024-08-22. 2024. URL: https://www.langchain.com/.

[23] LangChain. *LangChain: PDF Document Loaders*. Accessed: 2024-08-22. 2024. URL: https://python.langchain.com/v0.1/docs/modules/data_connection/document_loaders/pdf/.

[24] Seongyun Lee et al. *Aligning to Thousands of Preferences via System Message Generalization*. 2024. arXiv: 2405.17977 [cs.CL]. URL: https://arxiv.org/abs/2405.17977.

[25] Zachary Chase Lipton, Charles Elkan, and Balakrishnan Narayanaswamy. *Thresholding Classifiers to Maximize F1 Score*. 2014. arXiv: 1402.1892 [stat.ML]. URL: https://arxiv.org/abs/1402.1892.

[26] Jiacheng Liu et al. *Generated Knowledge Prompting for Commonsense Reasoning*. 2022. arXiv: 2110.08387 [cs.CL]. URL: https://arxiv.org/abs/2110.08387.

[27] Yiheng Liu et al. "Summary of ChatGPT-Related research and perspective towards the future of large language models". In: *Meta-Radiology* 1.2 (2023), p. 100017. ISSN: 2950-1628. DOI: https://doi.org/10.1016/j.metrad.2023.100017. URL: https://www.sciencedirect.com/science/article/pii/S2950162823000176.

[28] Igor Menzel et al. "An Experimental Comparison Regarding the Completeness of Functional Requirements Specifications". In: *2010 18th IEEE International Requirements Engineering Conference*. 2010, pp. 15–24. DOI: 10.1109/RE.2010.13.

[29] Meta AI. *LLaMA: Large Language Model as an Application*. https://ai.facebook.com/research/. Accessed: 2023-06-23. 2023.

[30] Microsoft. *Advanced Prompt Engineering - Azure OpenAI Service*. https://learn.microsoft.com/en-us/azure/ai-services/openai/concepts/advanced-prompt-engineering?pivots=programming-language-chat-completions#specifying-the-output-structure. Accessed: 2024-08-21. 2024.

[31] MongoDB. *Choosing a Chunking Strategy for Retrieval-Augmented Generation*. Accessed: 2024-08-22. 2024. URL: https://www.mongodb.com/developer/products/atlas/choosing-chunking-strategy-rag/.

[32] Emebo Onyeka et al. "Using Commonsense Knowledge and Text Mining for Implicit Requirements Localization". In: *2020 IEEE 32nd International Conference on Tools with Artificial Intelligence (ICTAI)*. 2020, pp. 935–940. DOI: 10.1109/ICTAI50040.2020.00146.

[33] OpenAI. *GPT-4: Overview and Capabilities*. https://www.openai.com/. Accessed: 2023-06-23. 2023.

[34] OpenAI. *Strategy: Write Clear Instructions*. Accessed: 2024-06-12. 2023. URL: https://platform.openai.com/docs/guides/prompt-engineering/strategy-write-clear-instructions.

[35] F. Paetsch, A. Eberlein, and F. Maurer. "Requirements engineering and agile software development". In: *WET ICE 2003. Proceedings. Twelfth IEEE International Workshops on Enabling Technologies: Infrastructure for Collaborative Enterprises, 2003.* 2003, pp. 308–313. DOI: 10.1109/ENABL.2003.1231428.

[36] Pinecone. *Chunking Strategies for Efficient Vector Search*. Accessed: 2024-08-22. 2024. URL: https://www.pinecone.io/learn/chunking-strategies/.

[37] Tobias Richter and Wolfgang Schnotz. "Textverstehen". In: Mar. 2018, pp. 826–837. ISBN: 3621282971.

[38] Alberto D. Rodriguez, Katherine R. Dearstyne, and Jane Cleland-Huang. "Prompts Matter: Insights and Strategies for Prompt Engineering in Automated Software Traceability". In: *2023 IEEE 31st International Requirements Engineering Conference Workshops (REW)*. 2023, pp. 455–464. DOI: 10.1109/REW57809.2023.00087.

[39] G. Sridhara, R. H. G., and S. Mazumdar. "ChatGPT: A Study on Its Utility for Ubiquitous Software Engineering Tasks". In: *arXiv* arXiv:2305.16837.[cs] (May 2023). [Online]. Available: http://arxiv.org/abs/2305.16837.

[40] Zhengxiang Wang, Jordan Kodner, and Owen Rambow. *Evaluating LLMs with Multiple Problems at once: A New Paradigm for Probing LLM Capabilities*. 2024. eprint: 2406.10786.

[41] Jason Wei et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. 2023. arXiv: 2201.11903 [cs.CL]. URL: https://arxiv.org/abs/2201.11903.

[42] Jason Wei et al. *Finetuned Language Models Are Zero-Shot Learners*. 2022. arXiv: 2109.01652 [cs.CL]. URL: https://arxiv.org/abs/2109.01652.

[43] Jules White et al. *A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT*. 2023. arXiv: 2302.11382 [cs.SE]. URL: https://arxiv.org/abs/2302.11382.

A. Appendix

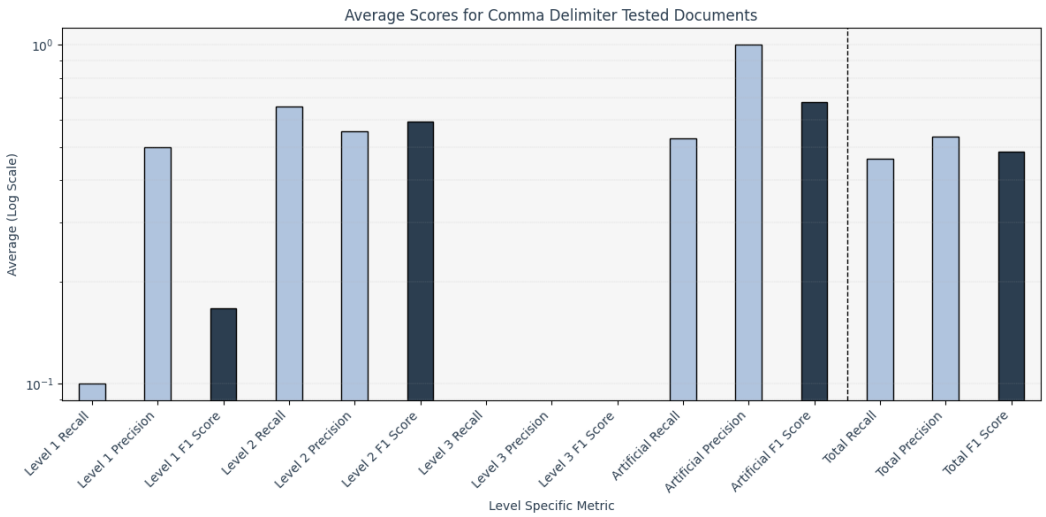

Figure A.1.: Coma Delimiter Level Scores

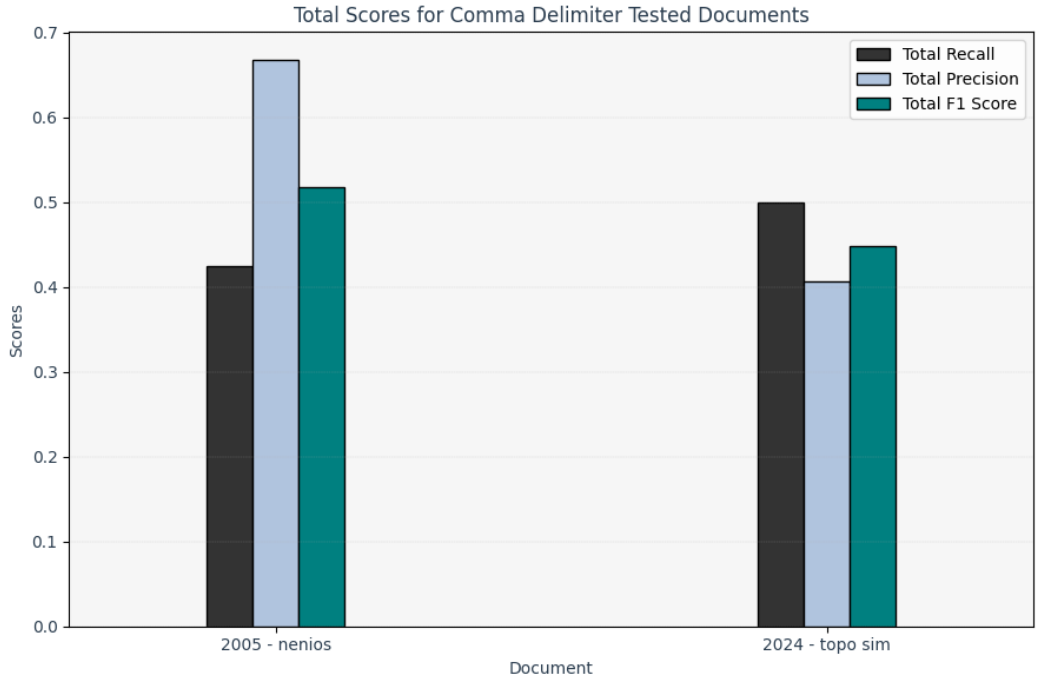

Figure A.2.: Coma Delimiter Total Scores

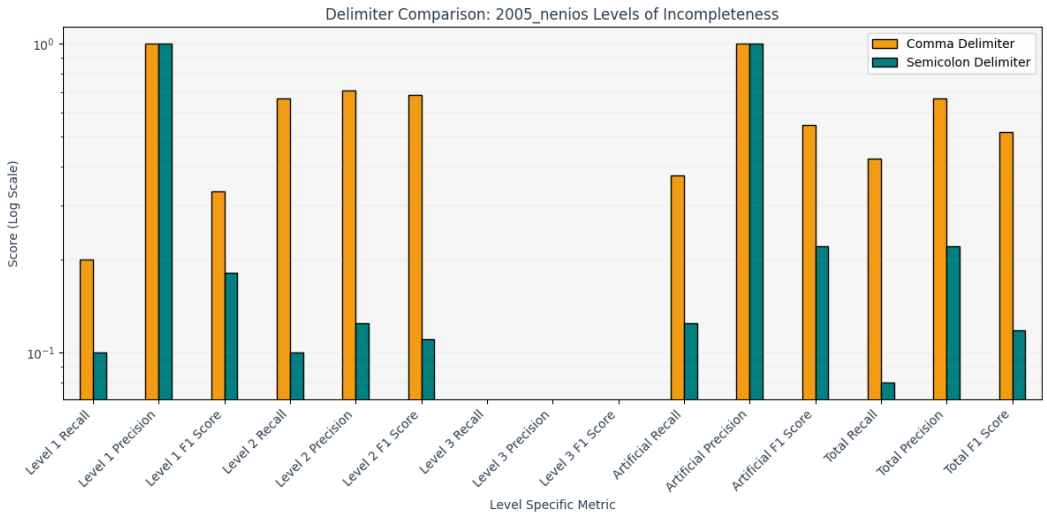

Figure A.3.: Comparing Delimiters in Dataset: 2005 - Nenios

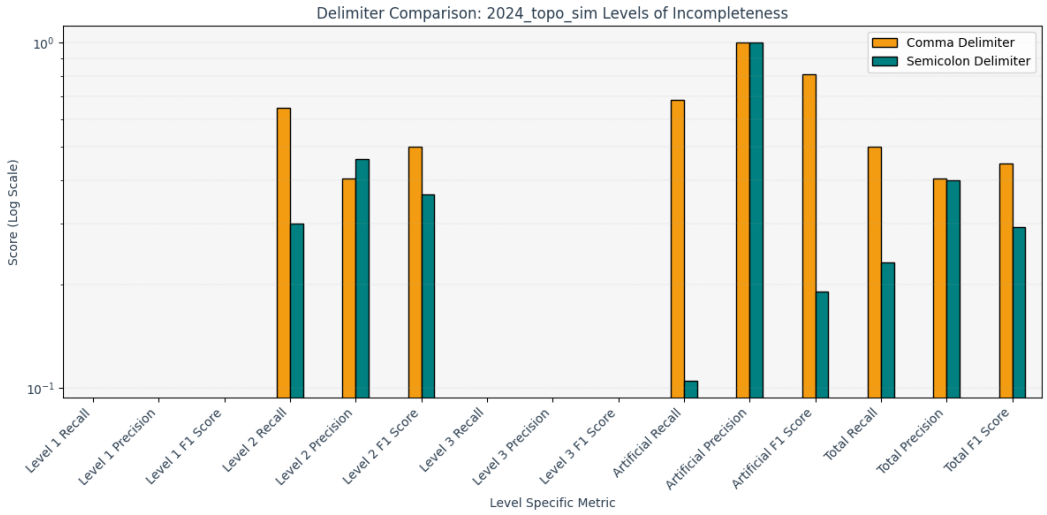

Figure A.4.: Comparing Delimiters in Dataset: 2024 - Topographical Projection Simulation

a)

	C1	C2	C3	C4	C5	C6
1	Eiffel Tower	Visit the iconic Eiffel Tower, one of the most recognizable landm…	Paris, France	1 day	Free to visit, elevator ticket €17	https://www.toureiffel.paris/en
2	Louvre Museum	Explore the world's largest and most famous museum, featuring an …	Paris, France	2-3 days	€18 adult admission	https://www.louvre.fr/en
3	Montmartre	Wander through the charming streets and alleys of this historic n…	Paris, France	Half day	Free to explore	https://www.parisinfo.com/discover-paris/neighborhoods/montmartre
4	Arc de Triomphe	Visit the iconic monument that honors the soldiers who fought and…	Paris, France	1 hour	€12 adult admission	https://www.arctriomphe.fr/en
5	Champs-Élysées	Stroll along this famous avenue lined with cafes, shops, and thea…	Paris, France	Half day	Free to walk	https://www.parisinfo.com/discover-paris/streets/champs-elysees
6	Notre-Dame Cathedral	Visit this beautiful Gothic cathedral, one of the most famous lan…	Paris, France	1 hour	Free to visit	https://www.notredamedeparis.fr/en
7	Palace of Versailles	Explore the opulent palace and gardens of the former royal residen…	Versailles, France	Full day	€20 adult admission	https://en.chateauversailles.fr/
8	Seine River Cruise	Take a romantic boat ride along the Seine River, offering stunnin…	Paris, France	2 hours	€15-20 per person	https://www.bateaux-mouches.com/en
9	Moulin Rouge	Experience the magic of this world-famous cabaret, featuring spec…	Paris, France	Evening	€100-200 per person	https://www.moulinrouge.fr/en
10	Palais Garnier	Visit the stunning opera house, known for its beautiful architect…	Paris, France	1 hour	€12 adult admission	https://www.operadeparis.fr/en
11	Montparnasse Tower	Enjoy panoramic views of the city from the top level of this 56-s…	Paris, France	1 hour	€9.50 adult admission	https://www.tourmontparnasse.com/en

b)

	C1	C2	C3
1	Rank	Activity	Description
2	1	Visit the Eiffel Tower	The iconic iron lady of Paris, offering breathtaking views of the city.
3	2	Explore the Louvre Museum	Home to the Mona Lisa and many other famous artworks from around the world.
4	3	Walk along the Seine River	Take a romantic stroll or boat ride along the City of Light's picturesque river.
5	4	Visit Notre-Dame Cathedral	A beautiful Gothic church that has stood for centuries, with stunning architecture and history.
6	5	Discover Montmartre	The charming neighborhood known for its bohemian vibe, street artists, and stunning views from the top.
7	6	Shop at Galeries Lafayette or Le Bon Marché	Indulge in luxury shopping and take in the stunning views of the city from these iconic department stores.
8	7	Visit the Arc de Triomphe	A monumental arch that honors the soldiers who fought and died for France, with great views of the city.
9	8	Enjoy a cabaret show at the Moulin Rouge or Lido	Experience the magic of Parisian nightlife with a classic cabaret performance.
10	9	Take a day trip to Versailles	Visit the opulent palace and gardens that were once home to French royalty, just outside the city.
11	10	Explore the trendy Marais neighborhood	Discover the hipster vibe, street art, and charming cafes in this up-and-coming Parisian district.

Figure A.5.: Comparing Delimiters in Ollama: a) above is coma, b) below is semicolon

Title	Priority	Reason
1999 - multi-mahjong	High	Fits Criteria
2001 - esa	High	Fits Criteria
2005 - nenios	High	Fits Criteria
2005 - phin	High	Fits Criteria
2009 - email	High	Fits Criteria
2009 - library	High	Fits Criteria
2009 - video search	High	Fits Criteria
2009 - warc III	High	Fits Criteria
2010 - blit draft	High	Fits Criteria
2010 - gparted	High	Fits Criteria
2010 - home 1.3	High	Fits Criteria
2010 - mashboot	High	Fits Criteria
0000 - inventory	Medium	Slightly Long
2001 - ctc network	Medium	Slightly Long
2004 - sprat	Medium	Slightly Long
2006 - stewards	Medium	Slightly Long
2009 - peazip	Medium	Slightly Long
2010 - split merge	Medium	Slightly Long
2005 - triangle	Medium	Slightly Fuzzy
2008 - keepass	Medium	Slightly Fuzzy
2009 - model manager	Medium	Slightly Fuzzy
2000 - cctns	Low	Too Fuzzy
1998 - themas	Low	Too Fuzzy
2002 - evla back	Low	Too Fuzzy
2002 - evla corr	Low	Too Fuzzy
2004 - grid bgc	Low	Too Fuzzy
2005 - znix	Low	Too Fuzzy
2007 - puget sound	Low	Too Fuzzy
2008 - peering	Low	Too Fuzzy
2009 - gaia	Low	Too Fuzzy
2003 - tachonet	Lowest	Too Long
2004 - rlcs	Lowest	Too Long
2005 - pontis	Lowest	Too Long
2007 - eirene fun 7	Lowest	Too Long
2007 - mdot	Lowest	Too Long
2007 - water use	Lowest	Too Long
2008 - vub	Lowest	Too Long
2010 - fishing	Lowest	Too Long

Table A.1.: Prioritization of Documents in PURE Dataset